Propuesta de Trabajo - Entrega II: Sistema de Gestión (Arquitectura Hexagonal)

Basado en los requerimientos de nuestra segunda entrega, he preparado esta propuesta para organizar nuestro trabajo. El reto principal es implementar Arquitectura Hexagonal (puertos y adaptadores) integrando 4 motores de base de datos distintos (PostgreSQL, MySQL, MongoDB y Firebase).

1. Análisis del Stack Tecnológico

Para el Frontend usaremos HTML, CSS (Tailwind vía CDN para agilizar) y JavaScript (Vanilla) usando fetch.

Para el Backend, necesitamos obligatoriamente un entorno de servidor para no exponer credenciales al cliente y aplicar correctamente la arquitectura hexagonal. Usaremos Node.js con Express.js.

Librerías principales del Backend:

- `pg` (PostgreSQL - Hospital General)
- `mysql2` (MySQL - Clínica Odontológica)
- `mongodb` (MongoDB - Laboratorio Clínico)
- `firebase-admin` (Firebase - Centro Pediátrico)
- `jsonwebtoken` y `bcrypt` (Autenticación y seguridad)
- `dotenv` (Manejo de variables de entorno)
- **Contratos de API y Estandarización de Respuestas (DTOs):** Para garantizar que el Frontend no dependa del motor de base de datos, el Backend implementará Data Transfer Objects (DTOs). Independientemente de si los datos provienen de una tabla relacional (PostgreSQL/MySQL) o de un documento NoSQL (MongoDB/Firebase), el Backend transformará la información para que el Frontend reciba siempre la misma estructura JSON. Esto se documentará previamente usando Swagger o Postman para que el equipo de Frontend conozca los contratos exactos antes de programar.
- **Seguridad y Control de Acceso Basado en Roles (RBAC):** Además de la autenticación por JWT, se implementarán middlewares de autorización. El token JWT incluirá el rol del usuario y su institución. El Backend validará que, por ejemplo, una "Secretaria" no pueda ejecutar endpoints de creación de diagnósticos, y que un "Médico" solo pueda operar sobre datos de su propia institución.

2. Distribución del Equipo (3 Personas)

Propongo la siguiente división para equilibrar la carga de trabajo:

- **Integrante A (Backend Core & Relacionales):** Encargado de la Arquitectura Hexagonal, lógica de dominio, autenticación, y los adaptadores de PostgreSQL (Hospital) y MySQL (Clínica).
- **Integrante B (Backend NoSQL & Datos):** Encargado de los adaptadores de MongoDB (Laboratorio) y Firebase (Pediátrico). Redactará las justificaciones de modelado documental (qué se embebe y qué se referencia) y el documento comparativo.
    - **Nota sobre la decisión de Modelado NoSQL:** Para justificar la decisión de qué se embebe y qué se referencia, el Integrante B se basará en el volumen y patrón de acceso a los datos:
        - **MongoDB (Laboratorio Clínico):** Se usará el enfoque de Referenciación. Dado que un paciente puede acumular cientos de exámenes a lo largo del tiempo, embeberlos en un solo documento del paciente superaría el límite de 16MB de MongoDB y degradaría el rendimiento. Se tendrán colecciones separadas para Pacientes, Órdenes y Resultados.
        - **Firebase (Centro Pediátrico):** Se usará el enfoque de Embebido (Subcolecciones). Los controles de crecimiento y el registro de vacunas se consultan siempre junto con el paciente y no suelen ser volúmenes masivos por visita. Se modelarán como subcolecciones dentro del documento del paciente (ej. pacientes/{id}/controles_crecimiento).
- **Integrante C (Frontend & QA):** Encargado de la interfaz gráfica (Login, Dashboards, Formularios), consumir los endpoints del backend, y liderar las pruebas de integración para asegurar que el frontend funcione exactamente igual sin importar el motor de BD.

3. Plan de Trabajo (Roadmap)

Fase 1: Configuración y Arquitectura

- GitHub: Crear el repositorio y configurar ramas (main, dev, feature/*).
- Definición de la Arquitectura Hexagonal (Dominio, Puertos/Interfaces y Adaptadores).
- Enrutamiento por Institución: Al hacer login, el backend identificará a qué institución pertenece el usuario y usará un Patrón Factory para instanciar el adaptador de BD correcto.

- **Definición de Contratos de API:** Los Integrantes A y B definirán los endpoints REST y la estructura JSON de respuesta unificada (DTOs) en Postman o Swagger.
- **Diseño de Claims del JWT:** Definir que el token de autenticación incluirá el id_institucion y el rol para que el middleware de enrutamiento y seguridad funcione.
- **Aprovisionamiento de Bases de Datos:** Levantar los entornos locales o en la nube, ejecutar los scripts .sql de la entrega 1 para PostgreSQL y MySQL, y configurar las colecciones iniciales y reglas de seguridad en MongoDB y Firebase.

Fase 2: Desarrollo Backend

- **Integrante A:** Configurar PostgreSQL y MySQL. Implementar lógica de consultas, diagnósticos y recetas. Implementar JWT.
- **Integrante B:** Modelado NoSQL (decidir qué se embebe y qué se referencia). Implementar CRUD en MongoDB y Firebase.
- **En conjunto:** Exponer endpoints REST unificados que sirvan para los 4 motores.
- **Implementación de RBAC:** Crear los middlewares de Express que validen los permisos según el rol del usuario antes de permitir operaciones de escritura (ej. bloqueo de diagnósticos a secretarias).
- **Patrón Factory en acción:** Implementar la lógica que lee el claim id_institucion del JWT y retorna la instancia del repositorio correcto (PostgreSQL, MySQL, MongoDB o Firebase).

Fase 3: Desarrollo Frontend e Integración

- **Integrante C:** Crear pantallas de Login, Dashboards y Formularios. Implementar comunicación con el backend vía fetch().
- **Regla de oro:** La interfaz no cambia según el motor; el backend traduce los datos para que el frontend reciba siempre la misma estructura JSON.
- **Manejo de Errores y Caídas de Servicio:** El frontend debe consumir los endpoints y estar preparado para mostrar alertas amigables si un motor de base de datos no responde (ej. "Servicio de laboratorio temporalmente no disponible"), sin romper la interfaz.

Fase 4: Pruebas, Documentación y Entrega

- **Buffer de Estabilización:** Reservar 2 días previos a la entrega exclusivamente para resolver bugs, hacer pruebas de estrés (envío de datos concurrentes a los 4 motores) y ensayar la presentación final.
- **Integrante C (QA):** Probar los 4 flujos completos cruzando instituciones y bases de datos. Documentar bugs.

- **Integrante B:** Redactar el Documento Comparativo exigido en la rúbrica (justificando por qué se usó cada motor para cada institución).
- **Todos:** Preparar el README.md y revisar el cumplimiento de la rúbrica (ej. archivar pacientes, historiales, etc.).

4. Consideraciones Técnicas Adicionales

1. **Variables de Entorno (.env):** Nunca subir las contraseñas a GitHub. Usar un `.env.example` de referencia.
2. **Transacciones:** Usar BEGIN, COMMIT, ROLLBACK en SQL (y batched writes en Firebase) para evitar que se guarde información a medias (ej. guardar consulta pero no receta).
3. **Seeders:** Crear scripts para insertar datos de prueba automáticamente.
4. **Documentación API:** Usar Postman o Swagger para definir qué JSON se enviará/recibirá.
5. **Desactivación de Historiales:** Asegurar que el backend bloquee la creación de nuevas citas si el paciente está "Archivado" o "Inactivo".
6. **Transacciones en NoSQL:** En PostgreSQL y MySQL se usarán transacciones estrictas (BEGIN, COMMIT, ROLLBACK) para asegurar que una consulta, su diagnóstico y su receta se guarden como una unidad atómica. En MongoDB se usarán transacciones multi-documento (disponibles desde la versión 4.0) para garantizar la consistencia entre una orden y su resultado. En Firebase se usarán Batched Writes para agrupar operaciones de escritura y asegurar que se completen todas o ninguna.
7. **Idempotencia en las peticiones:** Para evitar la duplicación de registros médicos si el usuario hace doble clic en "Guardar", el backend implementará validaciones en los endpoints POST (ej. verificar que no exista una consulta idéntica en un margen de tiempo corto).

5. Checklist de la Rúbrica

- ☐ Aplicación funcional cliente-servidor.
- ☐ Autenticación y enrutamiento por institución funcional.
- ☐ CRUD completo funcionando en los 4 motores de bases de datos.
- ☐ Opción de archivar pacientes habilitada y validada en negocio.
- ☐ Modelado NoSQL justificado (Embebido vs Referenciado).
- ☐ Documento comparativo de motores redactado.
- ☐ Repositorio en GitHub listo con commits claros.
- ☐ Contratos de API documentados y respetados por el frontend (misma estructura JSON para todos los motores).
- ☐ Middlewares de autorización por roles (RBAC) implementados y probados.

- [ ] Scripts de inicialización de Bases de Datos ejecutados sin errores.
- [ ] Manejo de errores en el frontend ante caídas de algún motor de BD.
- [ ] Pruebas de transacciones (SQL y NoSQL) exitosas sin dejar datos huérfanos.